AI Coding的下一站，不是更会写代码，而是更懂团队

原创 腾讯程序员 腾讯技术工程 Jul 30, 2026, 2:36 AM 广东 663 listened



作者：linjangyang、lypeerluo、kehawang、yansycao

> 个人 AI Coding 的效率提升已无悬念，但团队规模铺开后，一个更深的短板暴露了：每次 session 踩坑、纠偏产出的有效经验，session 结束即归零。业界在 Agent 经验管理上的探索多聚焦"个人记忆"，而要打破AI每次进项目都"从零开始"的循环，需要的是一套能持续捕获、提纯、沉淀团队经验，让AI带着项目语境进场的系统。本文分享从 0 到 1 建设这套系统的完整过程——从初版 90% 候选经验为废料的起点，到打磨出 Review/Dedup/Merge 三层治理链路，每一步踩在真实问题上的演化路径，以及驱动策略设计的实验数据和可复用方法论。

一、“别那么干，这个 hook 不能同步上报”

这是半年前的一次 CodeBuddy session。当时我们在开发一个 stop hook 的上报功能，AI 很自然地给出了同步上报的实现——合理，绝大多数上报场景同步就够了。我们 review 代码时改了：必须异步，否则 stop hook 会阻塞进程，后续操作直接超时。AI 根据反馈修正了代码，session 正常结束。

问题出在结束之后。

一周后，另一个同事的 session 触发了同样的场景。AI 没有那次 session 的上下文，给出的仍然是同步上报。同事不是那块代码的原始开发者，review 时没有发现这个陷阱。代码合入了，直到测试环境才暴露——stop hook 场景下进程阻塞，一连串超时错误。 这就是经验断层。不是 AI 不行——它在自己的 session 里被纠正过一次，做得很好。但那次纠正只在发起那轮 session 的开发者脑子里，AI 完全不记得发生过什么。

更普遍地看，我们团队在持续深入使用 AI Coding 一段时间后，陆续碰到四类这类问题：

- **经验即抛**：一次 session 反复纠偏踩出来的路径，session 结束就清零。下一个同事碰同样问题，AI 又是零基础起步。
- **重复踩坑**：项目的隐性约束——“这个目录下的代码访问第三方服务需要走特殊代理”“模块间开关引用不能直接用字符串参数，因为开关使用时机问题会导致永远默认关”——这些规则写在 git blame 里，但从来不进文档。换个人、换个 session，很大概率照踩不误。

- **知识碎片化**：老同事脑子里有一套"这么做才对"的 pattern，但没人有动力写下来。文档永远跟不上代码变更的速度。到了 AI 时代，靠的不是知识管理，是靠运气——这次 session 的上下文恰好覆盖了上次的历史，就对了；没覆盖，就错了。
- **AI 永远是"新同事"**：AI 不分项目。进新模块不知道架构边界在哪，不知道历史包袱是什么，也不知道团队踩过什么坑。它不缺编码能力，缺的是"这个团队的语境"。

所以整个事情的起点，不是"怎么让 AI 写代码更快"，而要打破AI每次进项目都"从零开始"的循环，这需要的是一套能持续捕获、提纯、沉淀团队经验，让AI带着项目语境进场的系统。

二、初版翻车：90% 的产出不可用

我们最初的方案非常直白。利用 CodeBuddy 的 Plugin Hook，上报所有的研发对话。然后让 AI 读这些对话，自动提取经验。Prompt 核心就一句话：**“提取你认为有价值的内容”**——边界的判断、排除规则，全交给模型。

逻辑上说得通：对话里反正有信息，模型也足够聪明。整条链路只有两步：

```
对话上报 → 经验抽取 → 入库
```

上线后我们很快发现，产出的经验里，大约 90% 是废经验，无法让后续工作更顺畅。这个"废"不是主观评价，是拿"被召回后 Agent 的行为是否发生正向改变"当标准来衡量的。而且废不是一种废，是四层结构性问题叠加在一起：

第一层：噪声极高。 原始对话包含大量调试过程——“继续”“重试一下”“不行，换个方式”——以及遍地的失败路径。这些不是经验，是过程录像。模型不加区分地全抽出来了。比如系统某次抽出一条"多试几次就好了"——这是在碰运气，不是在沉淀判断。

第二层：上下文脱离。 抽出来的经验脱离原始语境就失真了。比如有一次系统抽出"stop hook 需要异步处理"——听起来对，但实际上这条规则只适用于特定环节、特定场景。不加限定条件，会误导后续 Agent 在所有 hook 场景下无差别异步，反而搞出新的 bug。

第三层：错误引导。 比没经验更糟的，是错的经验。一条"超时就调大超时参数"的经验被抽出来进入库——根因不在这，这只是堆补丁。Agent 下次遇到类似问题，会优先调参数而不是排查根因，系统性走错路。

第四层：价值难定义。 LLM 的总结能力在开放域很强，但放在经验提取里，大量产出流于表面。“遇到复杂问题多收集上下文”——这句话永远对，但在具体工程场景下基本没有指导意义。AI 看完它不知道下一步该干嘛。

这四层问题暴露出一个核心事实：**过程录像不等于经验。自动提取放大候选集的同时，也在同步放大噪声。不加过滤的自动提取，就是一个垃圾放大器。** 让 AI 自己判断自己产出经验的价值，等于让裁判同时当运动员——在工程上无效。

三、为什么现成方案行不通

翻车之后，我们系统性地看了业界在"Agent 经验管理"方向上的方案。

ClaudeCode AutoDream 做的是单会话内自动记忆整理——ExtractMemories 抽取实体/偏好/状态，Auto Dream 异步合并。定位是"延续单 Agent 上下文"，把过程记忆当永久记忆存下来。它没有团队边界的意识，也不会区分什么值得长期留、什么只是临时状态。

Hermes Agent 是 Prompt 驱动的 memory 策略系统。它在特定条件（任务成功、踩坑解决）触发回顾——解决的是"何时记"，但解决不了"记的东西有没有价值"。缺乏系统级去重与来源追溯。

Mem0 是一套 Agent 长期记忆基础设施。ADD/UPDATE/DELETE/NONE 四类判定、混合检索，生命周期管理完整。但整个设计的假设是"记忆大体有价值"——这个假设在编程场景的高噪声对话里完全站不住。

把它们放在一起看，共同的局限是：**聚焦"个人记忆"，追求"记住更多"**。我们需要的是反向的：聚焦"团队经验质量"，追求"留下更少但更可信"——带适用场景、约束边界、来源证据的工程经验。 既然业界没有现成方案，我们只能自己摸索。



四、什么是"团队经验"——从 Agent 的视角重新定义

经验系统的最终服务对象不是人，是 Agent。Agent 的工作方式很明确：接收输入（包含召回的经验）→ 做出决策/产生输出。由此推导，判定一条信息是不是经验的标准只有一个：

被召回后，Agent 能否产生正向的行为变更。

它把"经验"从"一段有价值的文本"锚定到了"能否改变 Agent 行为"上，是一条可实际验证的工程标准。

基于这个标准，一条合格的团队经验必须同时满足几个条件：来自真实对话、有证据支撑、项目特有、不是通用常识、后续相似场景可复用。其中最关键的是 **“不容易直接发现”**——如果 Agent 自己就能推出来，召回它没有任何额外价值。但什么才算"不容易直接发现"？我们按 Agent 理解需求的路径拆成三类：

黑话镜头（语义不可发现）： 项目内部的黑话、缩写、代称，字面上完全无法推导。比如"D 站"在项目里指盗版站点、"A 站"指成人站点——AI 只看到字母，不知道背后语义。这类名称在对话里反复出现，但 AI 永远猜不对。

索引镜头（位置不可发现）： 某个工具、目录、能力入口不在直觉路径上。比如"直达页面功能在 xhome 模块，关键类是 FastCutXXX"——读代码找不到，需要知道"去哪找"。

逻辑镜头（行为不可发现）： 反直觉的工程约束、隐式机制。开头提到的"stop hook 不能同步上报"就是典型——AI 常规推理认为"上报是轻量操作，同步没问题"，但实际场景有并发阻塞风险。再比如"模块间开关引用不能直接用字符串参数"——开关确实有 String 参数，但用了之后由于开关使用时机的问题导致效果永远为默认关。这种坑，常规推断推不出来。

这三个镜头不是按主题（前端/后端）或重要性（高/中/低）分的——那些维度要么无穷无尽列不完，要么依赖主观判断不稳定。认知障碍是客观的：一条信息"Agent 能不能自己发现"，可以判定。

镜头类型	障碍本质	典型场景	信息缺口
黑话镜头	语义不可发现	"D 站"字面只是字母 D	需要显式映射为"盗版站点"
索引镜头	位置不可发现	“直达页面功能在哪”	需显式指向 xhome 模块 FastCutXXX
逻辑镜头	行为不可发现	stop hook 同步上报	AI 常规推理推不出并发阻塞风险

五、系统链路：被问题逼出来的演化

回到实际工程。初版的两步链路能跑通，但跑起来后问题一个一个暴露——每个都不是设计时预见的：

问题 1： 原始对话一股脑塞给模型，主题混杂、噪声大、上下文溢出。经验不是什么单轮对答就能判断的——需要看"失败 → 纠正 → 修复 → 采纳"的完整过程。

→ 新增 **主题分组** 环节：让模型结合 session 大纲按主题切分片段，分组后再逐个提取。 **问题 2**： 仅提取就直接入库，约 90% 是垃圾。

→ 新增 **Review 质量审核** 环节：入库前设一道质量闸口。 **问题 3**： 不同分组、不同 session 产生重复候选经验。

→ 新增 **Dedup 候选去重** 环节：在入库前识别并合并重复。 **问题 4**： 入库后无法长期维护，新经验与历史经验的关系是糊涂账。

→ 新增 **Merge 历史合并** 环节：判断新经验是新建、更新、跳过还是与历史冲突。

整条链路最终收敛为：

```
对话上报 → 主题分组 → 经验抽取 → Review → Dedup → Merge → 入库 → 召回统计
```

一个容易被忽视的关键是主题分组。我们试过两组对照：一组给分组后的片段额外附上 session 上下文大纲，一组不给。带大纲的那组产出了更多垃圾——因为大纲引导模型做泛化推断，反而偏离了分组片段里实际发生的具体经验。分组本身的粒度就够了。



六、三层治理：从"能跑"到"能用"

链路搭起来只是第一步。真正让系统可靠的是 Review / Dedup / Merge 三层治理。这三层目标不同、策略不同、默认方向也不同，但底层驱动方式一致——**错例分析 → 规则抽象 → 评测验证**。

6.1 抽取：从识别垃圾到评测驱动

在讲三层之前，得先说清楚源头——经验抽取这一步本身怎么持续变好。

初版证明了"能抽出经验"，但质量参差不齐。"好经验"很难一次定义到位，但"垃圾经验"的特征更容易归纳。所以我们的做法是 **先研究垃圾，再反推好经验**。经过大量标注，归纳出九类典型垃圾特征：

1. **事实性错误**——编造不存在的约束，把对话里的临时说法当规则
2. **通用常识**——任何项目都适用，不体现团队特异性
3. **对话摘要**——只复述过程，没有沉淀成可复用判断
4. **一次性 case**——只对当前任务有效，不能迁移
5. **用户主观偏好**——个人选择，不代表团队规范
6. **缺少上下文**——不知道对应哪个模块/组件/接口
7. **粒度混用**——一条经验里兼顾通用规则和 case 特定信息
8. **不可执行**——只有抽象建议，Agent 看完不知道怎么做
9. **证据不足**——对话里没有足够依据，模型自己推断过多

有了九类特征，走两条路径工程化：

标注路径： 对候选经验做价值标注（高价值/低价值/垃圾）→ 对垃圾做归因 → 归纳垃圾模式 → 转成标注规则和 Reviewer 标准。这条路径的意义是让"经验质量"从主观感觉变成可讨论、可对齐的对象。形成的评测集会持续用于后续所有 prompt 修改的效果测量。

可审计路径： 要求模型为每条经验输出三个维度的解释——为什么这是经验（解决了什么可复用问题、是否是项目特有）、命中了 prompt 中的哪些排除规则（通过了哪些垃圾检查）、对

话证据是什么（哪几轮对话支撑）。这让模型的判断过程脱离黑盒，也为后续 prompt 迭代提供了错例定位的依据。

有了这两条路径，Prompt 的迭代不再是"感觉不太好改一下试试"，而是有明确的结构化流程：

```
固定对话样本 → 当前 prompt 提取 → 与人工标注对齐 → 分析错例 → 修改 prompt → 重新评测 → 对比指标变化
```

三个核心指标一起看：Recall 看该提取的提了多少、Precision 看提取的有多少不是垃圾、Garbage Rate 看不该提取的提了多少。**不能只看一个**——只看 Precision 模型会过度保守，只看 Recall 垃圾会变多。目标是同时保持垃圾率下降和整体提取质量不退化。

6.2 Review：源码探索做事实性校验

Review 的定位是在入库前拦住明确垃圾。策略是**默认保留、定向过滤**——不追求全面收紧，只对明确的三类垃圾维度做精准拦截：事实性错误/偏好、缺上下文、粒度混用，外加一层无经验类别兜底。这一定位背后是风险判断：经验系统的首要风险是"漏掉好经验"大于"多放几条边缘经验"，默认保留能避免 Review 因过于严格误杀高价值内容。

Prompt 的裁决框架收敛为四步：事实性错误/偏好检查 → 缺上下文检查 → 粒度混用检查 → 无经验类别兜底。经过三轮迭代从"整体重构"到"边界强化"到"偏好规则硬化"，垃圾穿透不断改善。

但纯文本判断有一个天然局限：模型无法验证经验中提到的技术事实是否真实存在。

一条真实案例：系统抽出一条经验，声称 Android 列表开发中对接 FastScrollBar 应使用 attachToQBListView() 方法。从文字上看逻辑自洽——有具体场景、有方法名、有操作指引。纯文本 Review 大概率会放行。但我们引入了**源码探索**——在代码库中检索 FastScrollBar 的类定义，发现该类只有 attachToRecyclerView() 和 attach() 两个方法，根本不存在 attachToQBListView()。正确的方法是 FastScrollBarCompat.attachToQBRecyclerView()。经验被标记为事实性错误，直接拦截。

```
[Reviewer 候选实体] ──► 提取代码线索: "FastScrollBar", "attachToQBListView"
                                  |
                                  ▼ Code Explorer 源码定向搜索
[QQBrowser 源码库] ──► 匹配 Class "FastScrollBar" (收敛成功)
                                  |
                                  ▼ 成员函数级事实验证
        发现: 该类只有 attachToRecyclerView() 和 attach() 两个方法
              不存在 attachToQBListView()
        (正确方法属于 FastScrollBarCompat.attachToQBRecyclerView())
                                  |
                                  ▼
                    [ 触发 S1_FACTUAL_ERROR 拦截 ]
```

如果这条经验入库，Agent 在涉及列表的场景中会按指示调用一个不存在的方法，编译失败。更麻烦的是 Agent 会怀疑自己理解有误，而不是怀疑经验有问题——陷入反复重试的恶性循环。

6.3 Dedup：宁严勿宽，禁止桥接合并

Dedup 的定位是在候选经验集内部识别重复，减少后续流程中的冗余判断。这层的核心风险不是"漏去重"，而是误去重——把两条本应独立保留的经验错误合并，造成的边界污染是永久性的。

因此策略明确为**宁严勿宽**。几条严格否决规则：

- When 不同不能合——即使结论看起来相似，场景不同就是两条不同的经验
- 主结论类型不同不能合——操作建议、风险规避、排查方法之间不能混为一谈
- 局部子机制和完整系统经验不能合——粒度不一致，不能互相替代
- 同一技术事实但用途不同不能合——比如同一条 API 既用于性能优化又用于兼容处理，不能因为"都涉及这个 API"就合并

最关键的一条是**禁止桥接式合并**：A 和 B 在处理结果上有重叠、B 和 C 也相关，不能因此推断 A 和 C 是重复。经验的边界在于适用场景的约束条件，语义相似不等于经验等价——一旦桥接式合并把边界不同的经验串联归并，边界信息就永久丢失了。

在 260 条经验、4 个分组的评测中，整体 F1 为 **71.79%**。但更值得关注的指标是严格重复场景下的表现——What 和 When 均相同的经验，Recall 达到了 **91.67%**。这说明在最核心的去重目标上，策略是有效的。漏判主要集中在 What 为包含/相交、When 为不同/相交的边界模糊组合上——这类场景下策略更倾向于保护差异，避免贸然合并。

值得注意的是，这 260 条经验是分 4 批独立进行的，batch 之间表现差异明显。最极端的是 batch_004，仅识别出 6 个人工正例中的 1 个——该 batch 的正例全部落在非"双强一致"的组合上。这说明去重 prompt 对边界的敏感度在输入特征分布变化时仍有波动。

6.4 Merge：唯一目标先行，保护历史边界

Merge 是入库前的最后一关。它判断一条新经验和历史经验库之间的关系，执行四种动作之一：

- **create**：库中不存在同类目标经验，新建入库
- **update**：与历史经验同向，但带有实质新信息，合并更新
- **skip**：与新经验在核心结论上可互相替代，跳过
- **contradict**：核心结论直接冲突，标记为冲突并提人工裁决

策略上有三条主线：

唯一目标判定前置。 先从历史库中召回与新经验相关的子集，判断是否存在"唯一最合适的经验目标"。如果没有——比如新经验和几条历史经验都沾边，但都不属于同一条可维护经验

——**默认回退到 create**。这条规则从根本上避免了"强行匹配"——一条不够匹配的历史经验被硬推成 update 目标，污染边界。

Update 必须自检。 对每一笔 update 增加内部 quality_check：合并后的 When 是否过度泛化（丢失了原经验的场景约束）、What 是否保留了最具体、最安全、最可执行的操作建议、Why 是否保留了核心机制、是否引入了不受原始经验支持的新结论。Update 过判是当前最突出的问题—— Recall 高达 96.30% 但 Precision 只有 **78.79%**。也就是说大部分该 update 的都被识别了，但模型存在明显的"积极合并"倾向，把一些本该 create 的内容也判成了 update。

Contradict 不自动放过，不自动决断。 冲突意味着团队的新认知和旧认知出现了不一致——这本身就是有价值的信号，不应该被自动压制。系统只负责标记，具体的决策走人工裁决通道。

六轮实验、157 个统计样本中，整体 F1 达到 **94.27%**。Create 是最稳定的主类别（F1 **96.46%**），模型在"没有唯一目标就创建"上的判法已经比较可靠。Skip 的 Precision 达到 100%——一旦模型判成了 skip，基本都判对；但 Recall 只有 85.71%，仍有一部分真实 skip 被分流到了 update 或 create。Update 是改进空间最大的类别。



七、跑起来之后

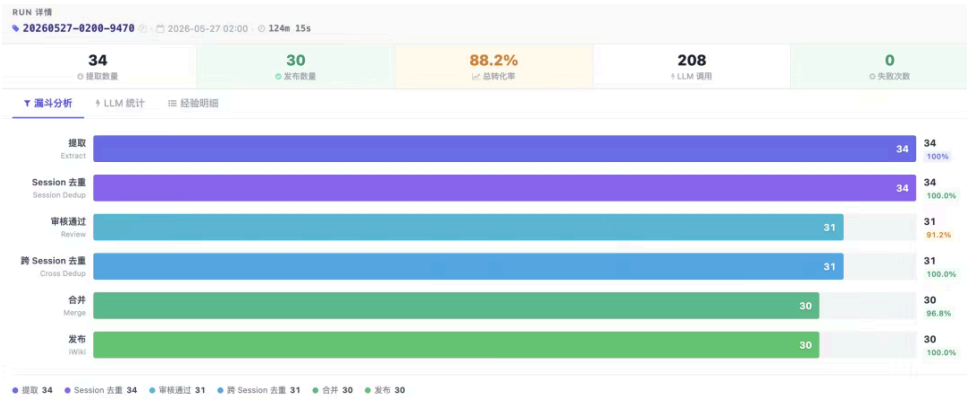
经过完整的三层治理漏斗，我们最终实现了从初版 90% 的抽取垃圾率到治理后 95% 的有效率，再到平均 80% 的最终入库率——大量的对话摘要、通用建议、一次性 case、缺乏上下文的碎片内容被扼杀在提取阶段、候选经验再被逐层拦截和合并。

截至目前，团队经验系统已在 QQ 浏览器团队 **6** 个仓库中常态化运行，覆盖 **50+** 名研发人员的日常开发 session，累计采集 **1,236** 次独立对话。从这些对话中累计提取出 **1,022** 条候选经验，经三层治理最终入库 **789** 条高置信经验，Pipeline 已稳定运行 **30+** 次。

7.1 完整的系统运行生命周期

以 2026/05/27 日这条 Pipeline 的真实运行案例来看系统如何运作。

本次处理原始对话记录数 39 条，初步提取候选经验 34 条，Review 后被拦截 3 条，与历史经验库合并 1条，最终入口 30 条：



其中一条 “*ForbiddenController 新增禁用能力须同步改三处*” 的候选经验被拦截，被识别的原因就是“在罗列修改清单，告诉读者去哪里找、改什么，而不是在说明隐藏技术事实。”，印证了 Review 门禁在精确区分"技术事实"这条红线上，动作准确。

而另一条 “*Chromium源代码层只能通过hooks层访问UBA功能*” 的候选经验则由于表达的是不易察觉的技术事实后果，因此被允许通过并入库。



在后续与 Agent 协作的开发场景里，我们通过 TDev 驱动需求实现的流程时，对话内部在特定阶段（探索）自动触发对历史经验的召回尝试：



7.2 召回消费：复用已有基建

在召回这里，我们没有重新造轮子，而是深度复用了公司已有的知识库平台：

1. **存储落库**：三层治理通过后的经验，自动写入 **IWIKI 经验空间**。
2. **自动索引**：**Knot 知识库系统**实时监听并自动索引 IWIKI 经验内容，生成向量与关键词索引。
3. **MCP 检索桥接**：Knot 提供标准 MCP 协议服务。Agent 在 session 期间自动调用 Knot MCP 接口检索相关经验并注入上下文。
4. **监控旁路**：在检索流程中增加了上报旁路，实时记录召回成功率和条数。

在最新 125 次真实开发检索的召回表现：请求级召回率**68.8%**，no_hit（未命中）仅 **4.8%**，平均每次注入 **2.4** 条经验，平均检索耗时 **1,299 ms**。

这种架构让团队将全部精力集中在最核心的"经验提纯与治理"上。

八、四条可复用的方法论

回看整个过程，有价值的不是某一个 prompt 怎么写、某一个参数怎么调，而是四条从反复踩坑中收敛出来的工程原则。

第一条：先定义资产边界，再做自动化沉淀。 初版 90% 废经验教会我们最核心的一课：不定义"什么算经验"就直接启动自动提取，系统会把对话摘要、通用建议、一次性 case 全塞进库。经验系统的关键不是"多提取"，而是"高置信地沉淀"。在工程化之前必须回答：什么算经验、什么不算、判断标准是什么。

第二条：分层治理，别用一个 prompt 解决所有问题。 Review 负责拦垃圾、Dedup 负责候选集内去重、Merge 负责历史库治理——每一层的目标不同、指标不同、prompt 设计也不

同。把它们揉在一起，各层目标就会互相牵制。用多个针对性模型各自完成一件事，比一个通用大模型一次性全搞定更可靠。

第三条：默认策略要按业务风险分别设计，没有统一答案。 Review 默认保留（误杀一条高价值经验的代价大于放行两条垃圾）、Dedup 宁严勿宽（误合并的边界污染不可逆）、Merge 保护历史边界（已有的经验是已验证的资产，新经验要跨过更高门槛才能修改它）。"宁可错杀"和"宁可放过"的方向在各层完全不同。

第四条：Prompt 优化要从错例中抽象规则，不是凭感觉改。 我们做 prompt 迭代不是"这个效果不好改一下试试"，而是固定的流程：收集错例 → 识别误判类型 → 抽象成可操作规则 → 多轮实验验证效果 → 判断规则是否有正向收益。同时用固定评测集防止数据泄露和过拟合。整个链路是工程化的，不是手艺活的。

结语

经验系统从 0 到 1 的建设已经跑通了——链路运转着，三层治理上线了，召回也在工作。但这只是起点。真正让经验系统从"能用"走到"可靠"，还有三个环环相扣的方向：

经验质量（生产阶段）： 当前三层治理生效之后，垃圾率降至约 5%。但事实性错误仍难以纯文本识别，源码探索只覆盖了部分场景；三个镜头之外的边界经验类型还没充分识别；评测集规模有限，长尾和冷门主题覆盖不全，prompt 迭代的边际收益在收窄。

使用效率（召回阶段）： 经验已能被召回并注入上下文，但命中不等于采纳——目前缺乏观测 Agent 是否真的依据经验改变行为的手段；召回侧的唯一目标判定偏粗，相似主题下容易出现"召回了但用不上"。

管理维护（生命周期）： 长期未命中、未采纳的经验没有自动淘汰机制，库会随时间静默膨胀；项目演进时（模块重构、能力下线）历史经验可能集体失效，目前没有自动识别"需要复核"的能力。

三者构成闭环：**生产决定基线** → **使用暴露问题** → **维护反哺生产**。任一环不闭合，经验系统就会从资产退回为噪声。

最后想说一句：经验系统的本质不是技术系统，是**团队认知能力的工程化管道**。过去经验在人脑里、产出在文档里；现在经验产生于人与 AI 的对话过程，需要一套管道来捕获、提纯、分发。管道一旦建成，团队就不再依赖"谁在那个时间段在做什么"来传承知识——AI 每次进入项目，都能带着"这个团队积累了什么、在什么场景下该怎么做"的语境。

QQ 浏览器平台技术团队

我们是腾讯 CSIG 旗下负责 QQ 浏览器平台研发的核心技术团队。致力于为超 4 亿用户打造极速、智能、全平台的浏览体验，产品覆盖 Android、iOS、鸿蒙、Windows、macOS 全终端。

团队深耕浏览器底层技术十余年，当前正深度参与 AI 在浏览器的能力落地，持续探索大模型时代技术的新边界。期待与更多同行切磋交流，共同成长。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容

公众号



阅读 1.4万

留言

写留言

还没有留言，来写首评